# A trace semantics for the process calculus of ProVerif

Artie Khovanov with lots of assistance from Claude Fable

ProVerif is a tool for reasoning about cryptographic security in a symbolic model with a Dolev-Yao adversary. Although it uses resolution of Horn clauses to find attacks, it is backed theoretically by a process calculus, which is a variant of the applied pi calculus. We use the specification of ProVerif's semantics given in Blanchet 16 [1].

We formalise the ProVerif process calculus in Lean. Rather than copying the specification, we use a shallow translation in order to make security proofs simpler down the line. This means that we make use of existing Lean syntax and constructs where possible. In particular, some invariants are enforced only by what terms can arise from translating a valid ProVerif program. We also deviate from the calculus when sticking exactly to the specification would complicate the design.

We use a trace-based semantics to encode the possible outcomes of parallel processes communicating on a network. The attacker is modelled as a set of traces satisfying informational constraints. Our trace semantics are good enough to express secrecy and simple ProVerif queries relating events. However, it is strictly less expressive than the formal operational semantics of ProVerif; in particular, we cannot express observational equivalence.

We give a mathematical account of the Lean development, and of the formal translation from terms in ProVerif's process calculus to our traces. Everything has been formalised in Lean.

## 1 The trace semantics

### 1.1 Messages

First, we define the set of messages. These are built from abstract constructors and subject to equations as well as destructor functions.

Fix a supply of variables $\{\nu_k \mid k \in \mathbb{N}\}$ and a family $\Sigma = (\Sigma_n)_{n\in\mathbb{N}}$ of arity-$n$ constructor symbols $f$. The set $\mathcal{T}$ of *constructor terms* $t$ is built up inductively by

$$t ::= \nu_k \mid f(t_1, \ldots, t_n),$$

where $f \in \Sigma_n$ $(n \in \mathbb{N})$.

In order to define deterministic destructors later, we must restrict the kinds of equations that may appear between constructor terms.

Fix a set of *free* constructors. A set of equations $E \subseteq \mathcal{T} \times \mathcal{T}$ is *valid* if, for every $(l, r) \in E$, both $l$ and $r$ are of the form $f(\ldots)$ with $f$ not free.

A *substitution* on terms is a homomorphic extension of a map $\{\nu_k\} \to \mathcal{T}$ on variables. Given a set of equations $E$, define $\equiv_E$ to be the least equivalence relation on constructor terms that contains $E$ and is closed under substitutions and constructor applications.

The free constructors are the ones which may be used when destructor patterns. In other words, the head symbols on either side of an equation cannot appear in any destructor's patterns.

**Definition 1.1** (Messages)**.** Fix everything as before. Given a valid set of equations $E$, the set of *messages* is $\mathcal{M} = \mathcal{T}/\equiv_E$.

The constants $\nu_k$ and constructors $f$ descend to $\mathcal{M}$ in the obvious way, and every substitution on constructor terms descends uniquely to one on messages.

Adjoin a symbol $\perp$ to $\mathcal{M}$ to represent the failure of a computation; for $g : \mathcal{M}^n \to \mathcal{M}$, write $g^\uparrow$ for the extension to $\mathcal{M}_\perp^n \to \mathcal{M}_\perp$ which returns $\perp$ when any argument is $\perp$.

We now come to destructors, whose definition is a bit fiddly. The details are ultimately unimportant to the overall model.

**Definition 1.2** (Destructor)**.** A *pattern* $p$ is given by

$$p ::= \nu_k \mid \perp \mid \lceil T \rceil,$$

where $T \in \mathcal{T}$. We extend a valuation $\theta : \{\nu_k\} \to \mathcal{M}_\perp$ to a pattern in the obvious way, except that we only define the extension to $\lceil T \rceil$ if $\theta(\nu_k) \neq \perp$ for all $\nu_k$ appearing in $T$. We say $\theta$ *matches* $p$ with $u \in \mathcal{M}_\perp$ if $\theta(p) = u$.

A *destructor* $g : \mathcal{M}_\perp^n \to \mathcal{M}_\perp$ is given by a list of rules $g(\vec{p}_1) \to q_1;\ \ldots;\ g(\vec{p}_k) \to q_k$ such that

1. each constructor on the left-hand side is free,
2. each variable on the right-hand side of a rule occurs on its left-hand side, with those under $\lceil \cdot \rceil$ on the right-hand side also occurring under $\lceil \cdot \rceil$ on the left-hand side.

As a function, $g$ is defined by

$$g(\vec{u}) = \begin{cases} \theta(q_i) & i \text{ least such that some } \theta \text{ matches } \vec{p}_i \text{ with } \vec{u}, \\ \perp & \text{otherwise.} \end{cases}$$

Destructors are well-defined because, by construction, there is at most one valuation matching a pattern against a (possibly failing) term.

Intuitively, the pattern $\nu_k$ represents a possibly-failing message $u \in \mathcal{M}_\perp$, $\lceil \nu_k \rceil$ represents a non-failing message $m \in \mathcal{M}$, $\perp$ represents $\perp$, and $\lceil f(t_1, \ldots, t_n) \rceil$ represents $f(m_1, \ldots, m_n) \in \mathcal{M}$ such that repeated variables are substituted the same.

**Definition 1.3** (Public operations)**.** Fix a set of public constructors $\mathrm{Pub} \subseteq \bigcup_n \Sigma_n$ and public destructors Dt. The *public operations* $\mathcal{M}_\perp^n \to \mathcal{M}_\perp$ are $\mathrm{Ops} = \{f^\uparrow \mid f \in \mathrm{Pub}\} \cup \mathrm{Dt}$. Write $\langle S \rangle$ for the closure of $S \subseteq \mathcal{M}$ under public operations (possibly involving failing terms).

Our adversary will grow its knowledge by applying public operations to the messages it knows. Note that this automatically includes public constant constructors.

## 1.2 Processes and their traces

In our algebraic model of cryptographic protols, processes run computations on messages in parallel and exchange them over a hostile network. We represent a process by its possible traces: sequences of events which characterise the security properties of that run.

**Definition 1.4** (Trace)**.** An *action* is given by

$$a \coloneqq \mathsf{out}(c, m) \mid \mathsf{in}(c, m) \mid \mathsf{comm}(c, m) \mid \mathsf{new}(\nu_k) \mid \mathsf{ev}(m),$$

where $c, m \in \mathcal{M}$ and $k \in \mathbb{N}$.

A *trace* is list of actions; write $\varepsilon$ for the empty trace and and $s \leq t$ if $s$ is a prefix of $t$. A trace is *closed* if it does not contain any "unfinished" actions $\mathsf{out}(c, m)$ or $\mathsf{in}(c, m)$. A *process* is a prefix-closed set of actions.

Write Act for the set of actions, Tr for the set of traces, and Proc for the set of processes.

The actions $\mathsf{in}$ and $\mathsf{out}$ represent one side of a message transmission, while $\mathsf{comm}$ represents a completed communication between parallel sub-processes. The action $\mathsf{new}$ represents the creation of a nonce, and is necessary to prevent nonce collision when running processes in parallel. The action $\mathsf{ev}$ allows a process to raise an arbitrary event flag – it's used to signal things like key leaks.

**Definition 1.5** (Process constructors)**.** We define the following standard processes. For an action $a$, a process $P$, and a map $F : \mathcal{M} \to \mathrm{Proc}$:

$$\begin{aligned} \mathbf{0} &= \{\varepsilon\}, & a \cdot P &= \{\varepsilon\} \cup \{a\, t \mid t \in P\}, \\ \mathsf{out}(c, m).P &= \mathsf{out}(c, m) \cdot P, & \mathsf{ev}(m).P &= \mathsf{ev}(m) \cdot P, \\ \mathsf{in}(c).F &= \textstyle\bigcup_{m \in \mathcal{M}} \mathsf{in}(c, m) \cdot F(m), & \mathsf{new}.F &= \textstyle\bigcup_{k \in \mathbb{N}} \mathsf{new}(k) \cdot \{t \in F(\nu_k) \mid k \notin \mathrm{nonces}(t)\}. \end{aligned}$$

By construction, the sets of traces remain prefix-closed. A trace should be interpreted as a **partial** execution of a process in an arbitrary network: arbitrary messages may be received, and outgoing messages may be intercepted. It remains to define parallel composition of processes, which is a bit more involved due to the need to keep track of completed communications.

Fix an index set $I$. Given a "tagged trace" $s \in (I \times \mathrm{Act})^*$ and $i \in I$, write $s{\restriction}i \in \mathrm{Tr}$ for the sub-trace of actions tagged $i$ and $\bar{s} \in \mathrm{Tr}$ for the trace with the tags forgotten. For a family of traces $\vec{t} : I \to \mathrm{Tr}$, the tagged trace $s$ *interleaves* the $t_i$ if $s{\restriction}i = t_i$ for all $i$.

Rather than simply forgetting the tags, we say a tagged trace $s$ *reduces to* the trace $t$ if it satisfies the inductively-given relation

$$\begin{gathered} s \to \bar{s}, \\ s \to t, i \neq j \Rightarrow (i, \mathsf{out}(c, m))\,(j, \mathsf{in}(c, m))\, s \to \mathsf{comm}(c, m)\, t. \end{gathered}$$

**Definition 1.6** (Parallel composition)**.** Given a family $P : I \to \mathrm{Proc}$ of processes, a trace $t$ lies in their parallel composition $\mathrm{merge}_I(P)$ if it has no repeated occurences of $\mathsf{new}(\nu_k)$ and there is a tagged trace $s$ and a family of traces trace $\vec{u}$ such that $u_i \in P_i$ for all $i$, $s$ interleaves $\vec{u}$, and $s$ reduces to $t$.

We write $P \mid Q = \mathrm{merge}_{\{0,1\}}(0 \mapsto P,\ 1 \mapsto Q)$ and $!P = \mathrm{merge}_{\mathbb{N}}(k \mapsto P)$. We can prove that $!P = P \mid !P$.

A trace in the parallel composition of several processes represents one possible parallel execution of these processes. Keep in mind that, since messages may be intercepted on the network, a communication between two parallel process may, or may not, be completed.

### 1.3 Translation of processes

**Definition 1.7** (Source syntax)**.** Fix a set of variables $x, y, \ldots$ and a set of names $a, b, c, \ldots$ containing the free names $\mathcal{N}_{\mathrm{pub}} \uplus \mathcal{N}_{\mathrm{priv}}$. Terms, expressions and processes are generated by

$$\begin{aligned} M, N &::= x \mid a \mid f(M_1, \ldots, M_n), \\ D &::= M \mid h(D_1, \ldots, D_n) \mid \mathsf{fail}, \\ P, Q &::= \mathbf{0} \mid \mathsf{out}(N, M); P \mid \mathsf{in}(N, x); P \mid P \mid Q \mid !P \mid \mathsf{new}\ a; P \\ &\quad \mid \mathsf{let}\ x = D\ \mathsf{in}\ P\ \mathsf{else}\ Q \mid \mathsf{if}\ M\ \mathsf{then}\ P\ \mathsf{else}\ Q \mid \mathsf{event}(M); P, \end{aligned}$$

where $f$ ranges over constructors and $h$ over constructors and destructors. Types are erased.

**Definition 1.8** (Model)**.** A *model* consists of a signature $\Sigma$ whose nullary symbols include $\mathcal{N}_{\mathrm{pub}} \uplus \mathcal{N}_{\mathrm{priv}}$, a theory $(E, \mathrm{Fr})$ over $\Sigma$, a finite set of destructors each given by rules as in Definition 1.2, sets Pub, Dt of public constructors and public destructors, and a closed process $P_0$ with $\mathrm{fn}(P_0) \subseteq \mathcal{N}_{\mathrm{pub}} \uplus \mathcal{N}_{\mathrm{priv}}$.

**Definition 1.9** (Translation)**.** An *environment* $\rho$ maps variables and names to $\mathcal{M}$; $\rho_0$ maps each free name $a$ to the constant $a \in \mathcal{M}$. Write $\rho M \in \mathcal{M}$ for the homomorphic extension. Expressions are evaluated in $\mathcal{M}_\perp$:

$$[\![M]\!]\rho = \rho M, \qquad [\![\mathsf{fail}]\!]\rho = \perp, \qquad [\![f(\vec{D})]\!]\rho = f^{\uparrow}([\![\vec{D}]\!]\rho), \qquad [\![g(\vec{D})]\!]\rho = g([\![\vec{D}]\!]\rho)$$

for a constructor $f$ and a destructor $g$. Processes are translated by

$$\begin{aligned} [\![\mathbf{0}]\!]\rho &= \mathbf{0}, & [\![\mathsf{out}(N, M); P]\!]\rho &= \mathsf{out}(\rho N, \rho M).[\![P]\!]\rho, \\ [\![\mathsf{event}(M); P]\!]\rho &= \mathsf{ev}(\rho M).[\![P]\!]\rho, & [\![\mathsf{in}(N, x); P]\!]\rho &= \mathsf{in}(\rho N).\big(m \mapsto [\![P]\!]\rho[x \mapsto m]\big), \\ [\![P \mid Q]\!]\rho &= [\![P]\!]\rho \mid [\![Q]\!]\rho, & [\![\mathsf{new}\ a; P]\!]\rho &= \mathsf{new}.\big(m \mapsto [\![P]\!]\rho[a \mapsto m]\big), \\ [\![!P]\!]\rho &= !\,[\![P]\!]\rho, & [\![\mathsf{let}\ x = D\ \mathsf{in}\ P\ \mathsf{else}\ Q]\!]\rho &= \begin{cases} [\![P]\!]\rho[x \mapsto m] & [\![D]\!]\rho = m \in \mathcal{M}, \\ [\![Q]\!]\rho & [\![D]\!]\rho = \perp, \end{cases} \\ & & [\![\mathsf{if}\ M\ \mathsf{then}\ P\ \mathsf{else}\ Q]\!]\rho &= \begin{cases} [\![P]\!]\rho & \rho M = \mathsf{true}, \\ [\![Q]\!]\rho & \rho M \neq \mathsf{true}. \end{cases} \end{aligned}$$

The translation of $P_0$ is $[\![P_0]\!]\rho_0$. A translated process mentions no $\nu_k$ except through $\mathsf{new}$.

*Remark* 1.10. In the development a process is a program in the continuation monad with answer type Proc, run by $\mathrm{run}(b) = b(\lambda_.\,\mathbf{0})$, with primitives $\mathrm{send}(c, m) = \lambda\kappa.\,\mathsf{out}(c, m).\kappa()$, $\mathrm{recv}(c) = \lambda\kappa.\,\mathsf{in}(c).\kappa$, $\mathrm{fresh} = \lambda\kappa.\,\mathsf{new}.\kappa$, $\mathrm{emit}(m) = \lambda\kappa.\,\mathsf{ev}(m).\kappa()$, $\mathrm{fork} = \lambda\kappa.\,\kappa(\mathrm{tt}) \mid \kappa(\mathrm{ff})$, $\mathrm{repl} = \lambda\kappa.\,!\kappa()$ and $\mathrm{stop} = \lambda_.\,\mathbf{0}$; let and if are case analysis in the metalanguage. The clauses above are the unfolding of that presentation.

### 1.4 The attacker and the security properties

**Definition 1.11** (Enemy)**.** The processes $\mathrm{Enemy}(K) \in \mathrm{Proc}$, for $K \subseteq \mathcal{M}$, are the pointwise least such that

1. $\varepsilon \in \mathrm{Enemy}(K)$;
2. if $K \vdash c$ and $e \in \mathrm{Enemy}(K \cup \{m\})$ then $\mathsf{in}(c, m)\, e \in \mathrm{Enemy}(K)$, for every $m \in \mathcal{M}$;
3. if $K \vdash c$, $K \vdash m$ and $e \in \mathrm{Enemy}(K)$ then $\mathsf{out}(c, m)\, e \in \mathrm{Enemy}(K)$;
4. if $K \vdash c$, $K \vdash m$ and $e \in \mathrm{Enemy}(K)$ then $\mathsf{comm}(c, m)\, e \in \mathrm{Enemy}(K)$, an output to itself;
5. if $k \notin \mathrm{nonces}(e)$ and $e \in \mathrm{Enemy}(K \cup \{\nu_k\})$ then $\mathsf{new}(k)\, e \in \mathrm{Enemy}(K)$.

**Definition 1.12** (Attacks, knowledge)**.** For $P \in \mathrm{Proc}$,

$$\mathrm{Attacks}(P) = \{t \in P \mid \mathrm{Enemy}(\emptyset) \ :\ \mathrm{Closed}(t)\}.$$

For $K \subseteq \mathcal{M}$, $\mathrm{known}(K, \cdot) : \mathrm{Tr} \to \mathcal{P}(\mathcal{M})$ is defined by

$$\begin{aligned} \mathrm{known}(K, \varepsilon) &= K, \\ \mathrm{known}(K, a\, t) &= \mathrm{known}\big(\langle K \cup \{m \mid a = \mathsf{comm}(c, m),\ c \in K\}\rangle,\ t\big). \end{aligned}$$

**Proposition 1.13** (Prefix-closure)**.** $\mathrm{Enemy}(K)$ *and every process built from the constructors of Definitions 1.5 and 1.6 are prefix-closed. If $P$ is prefix-closed, so is* $\mathrm{Attacks}(P)$.

**Definition 1.14** (Facts, secrecy, correspondence)**.** A *fact* is $\mathsf{event}(e)$ or $\mathsf{knows}(m)$ with $e, m \in \mathcal{M}$; Fact is the set of facts. A fact *holds* after $t \in \mathrm{Tr}$, written $t \models f$, if

$$t \models \mathsf{event}(e) \iff \mathsf{ev}(e) \in t, \qquad t \models \mathsf{knows}(m) \iff m \in \mathrm{known}(\langle\emptyset\rangle, t).$$

For a set $X$, $F, H : (X \to \mathcal{M}) \to \mathrm{Fact}$ and $s \in \mathcal{M}$,

$$\begin{aligned} \mathrm{Corr}(P, F, H) \iff\ & \forall \sigma : X \to \mathcal{M}.\ \forall t \in \mathrm{Attacks}(P). \\ & t \models F(\sigma) \Rightarrow \exists \sigma' : X \to \mathcal{M}.\ F(\sigma') = F(\sigma) \wedge t \models H(\sigma'), \\ \mathrm{Secret}(P, s) \iff\ & \forall t \in \mathrm{Attacks}(P).\ t \not\models \mathsf{knows}(s). \end{aligned}$$

By Proposition 1.13, for prefix-closed $P$ these are properties of every prefix of every attack: $\mathrm{Corr}(P, F, H)$ says that some $H(\sigma')$ holds by the time $F(\sigma)$ does.

**Definition 1.15** (Queries)**.** A *query fact* is $\mathsf{event}(M)$ or $\mathsf{attacker}(M)$ with $M$ a term over constructors, free names and a finite set $X$ of variables; it is an *event fact* or an *attacker fact* accordingly. For a valuation $\sigma : X \to \mathcal{M}$ write $\sigma M \in \mathcal{M}$ for the evaluation of $M$ with free names as constants, and let the *instance* of a query fact under $\sigma$ be $\sigma\, \mathsf{event}(M) = \mathsf{event}(\sigma M)$ and $\sigma\, \mathsf{attacker}(M) = \mathsf{knows}(\sigma M)$. The query $\mathsf{query\ attacker}(M)$, for $M$ without variables, denotes $\mathrm{Secret}([\![P_0]\!]\rho_0, \rho_0 M)$. The query $\mathsf{query}\ F \Longrightarrow H$, with $F$ and $H$ query facts over $X$, denotes $\mathrm{Corr}([\![P_0]\!]\rho_0,\ \sigma \mapsto \sigma F,\ \sigma \mapsto \sigma H)$, the survey's Definition 3.5 with $=$ read as $=_E$.

## 2 Relation to the operational semantics of ProVerif

### 2.1 The reduction semantics

This section restates the semantics of Blanchet's survey (§2.4, §2.5.1, §3.1–3.3) for the syntax of Definition 1.7. Names are atoms; substitutions map variables to terms and never substitute $\mathsf{fail}$ or a may-fail variable for an ordinary variable. An equational theory $E$ is a finite set of equations between terms without names; $=_E$ is the closure of $E$ under reflexivity, symmetry, transitivity, application of function symbols and substitution. Each constructor $f$ carries the rule $f(\vec{x}) \to f(\vec{x})$, each destructor $g$ a finite ordered list of rules $g(\vec{U}) \to U$ between may-fail terms without names, and every $\mathrm{def}(h)$ ends with $h(\vec{u}) \to \mathsf{fail}$ for may-fail variables $\vec{u}$. The survey states §3 for the core calculus; here its definitions are taken modulo $=_E$: a step on a term $M' =_E M$ counts as a step on $M$, and substitutions are compared modulo $=_E$.

Evaluation $D \Downarrow U$ of closed expressions to closed may-fail terms is the least relation with $M \Downarrow M$, $\mathsf{fail} \Downarrow \mathsf{fail}$, and $h(D_1, \ldots, D_n) \Downarrow \sigma U'_j$ whenever $D_i \Downarrow U_i$ for all $i$, $\mathrm{def}(h) = (h(U'_{i,1}, \ldots, U'_{i,n}) \to U'_i)_{i \le k}$, $\sigma U'_{j,i} =_E U_i$ for all $i$, and for all $i < j$ and all $\sigma'$ there is some $l$ with $\sigma' U'_{i,l} \neq_E U_l$.

A *configuration* is $(\mathcal{N}_{\mathrm{pub}}, \mathcal{N}_{\mathrm{priv}}), \mathcal{P}$ with $\mathcal{N}_{\mathrm{pub}}, \mathcal{N}_{\mathrm{priv}}$ finite disjoint sets of names and $\mathcal{P}$ a finite multiset of closed processes with $\mathrm{fn}(\mathcal{P}) \subseteq \mathcal{N}_{\mathrm{pub}} \cup \mathcal{N}_{\mathrm{priv}}$. Writing $\mathcal{E}$ for the pair of name sets, reduction is the least relation closed under

$$\mathcal{E}, \mathcal{P} \cup \{\mathbf{0}\} \to \mathcal{E}, \mathcal{P} \qquad \text{(Red Nil)}$$

$$\mathcal{E}, \mathcal{P} \cup \{P \mid Q\} \to \mathcal{E}, \mathcal{P} \cup \{P, Q\} \qquad \text{(Red Par)}$$

$$\mathcal{E}, \mathcal{P} \cup \{!P\} \to \mathcal{E}, \mathcal{P} \cup \{P, !P\} \qquad \text{(Red Repl)}$$

$$(\mathcal{N}_{\mathrm{pub}}, \mathcal{N}_{\mathrm{priv}}), \mathcal{P} \cup \{\mathsf{new}\ a; P\} \to (\mathcal{N}_{\mathrm{pub}}, \mathcal{N}_{\mathrm{priv}} \cup \{a'\}), \mathcal{P} \cup \{P\{a'/a\}\} \quad \text{if } a' \notin \mathcal{N}_{\mathrm{pub}} \cup \mathcal{N}_{\mathrm{priv}} \qquad \text{(Red Res)}$$

$$\mathcal{E}, \mathcal{P} \cup \{\mathsf{out}(N, M); Q,\ \mathsf{in}(N', x); P\} \to \mathcal{E}, \mathcal{P} \cup \{Q,\ P\{M/x\}\} \quad \text{if } N =_E N' \qquad \text{(Red I/O}')$$

$$\mathcal{E}, \mathcal{P} \cup \{\mathsf{let}\ x = D\ \mathsf{in}\ P\ \mathsf{else}\ Q\} \to \mathcal{E}, \mathcal{P} \cup \{P\{M/x\}\} \quad \text{if } D \Downarrow M \qquad \text{(Red Eval 1)}$$

$$\mathcal{E}, \mathcal{P} \cup \{\mathsf{let}\ x = D\ \mathsf{in}\ P\ \mathsf{else}\ Q\} \to \mathcal{E}, \mathcal{P} \cup \{Q\} \quad \text{if } D \Downarrow \mathsf{fail} \qquad \text{(Red Eval 2)}$$

$$\mathcal{E}, \mathcal{P} \cup \{\mathsf{if}\ M\ \mathsf{then}\ P\ \mathsf{else}\ Q\} \to \mathcal{E}, \mathcal{P} \cup \{P\} \quad \text{if } M =_E \mathsf{true} \qquad \text{(Red Cond 1}')$$

$$\mathcal{E}, \mathcal{P} \cup \{\mathsf{if}\ M\ \mathsf{then}\ P\ \mathsf{else}\ Q\} \to \mathcal{E}, \mathcal{P} \cup \{Q\} \quad \text{if } M \neq_E \mathsf{true} \qquad \text{(Red Cond 2}')$$

$$\mathcal{E}, \mathcal{P} \cup \{\mathsf{event}(M); P\} \to \mathcal{E}, \mathcal{P} \cup \{P\} \qquad \text{(Red Event)}$$

A *trace* is a finite reduction sequence $\mathcal{E}_0, \mathcal{P}_0 \to \cdots \to \mathcal{E}_n, \mathcal{P}_n$, written $\mathcal{E}_0, \mathcal{P}_0 \to^* \mathcal{E}_n, \mathcal{P}_n$.

**Definition 2.1** (Survey, Definitions 3.1–3.5)**.** Let $P_0$ be a closed process.

1. A closed process $Q$ is an $\mathcal{N}_{\mathrm{pub}}$-*adversary* if $\mathrm{fn}(Q) \subseteq \mathcal{N}_{\mathrm{pub}}$, every function symbol of $Q$ is public, and $Q$ contains no $\mathsf{event}$.
2. A trace *outputs $M$ publicly* if it contains a step (Red I/O$'$) whose output is $\mathsf{out}(N, M'); Q$ with $N \in \mathcal{N}_{\mathrm{pub}}$ and $M' =_E M$. It *executes* $\mathsf{event}(M)$ if it contains a step (Red Event) on $\mathsf{event}(M'); P$ with $M' =_E M$.
3. $P_0$ *preserves the secrecy of* the closed term $M$ *from* $\mathcal{N}_{\mathrm{pub}}$ if, for some $\mathcal{N}_{\mathrm{priv}}$ disjoint from $\mathcal{N}_{\mathrm{pub}}$ with $\mathrm{fn}(M) \cup \mathrm{fn}(P_0) \subseteq \mathcal{N}_{\mathrm{pub}} \cup \mathcal{N}_{\mathrm{priv}}$, no trace $(\mathcal{N}_{\mathrm{pub}}, \mathcal{N}_{\mathrm{priv}}), \{P_0, Q\} \to^* \mathcal{E}', \mathcal{P}'$ with $Q$ an $\mathcal{N}_{\mathrm{pub}}$-adversary outputs $M$ publicly.

4. $P_0$ *satisfies* $\mathsf{event}(M) \rightsquigarrow \bigvee_{j=1}^{m} \bigwedge_{k=1}^{l_j} \mathsf{event}(M_{jk})$ *against* $\mathcal{N}_{\mathrm{pub}}$*-adversaries* if, for some $\mathcal{N}_{\mathrm{priv}}$ as above, for every $\mathcal{N}_{\mathrm{pub}}$-adversary $Q$, every trace Tr from $(\mathcal{N}_{\mathrm{pub}}, \mathcal{N}_{\mathrm{priv}}), \{P_0, Q\}$ and every substitution $\sigma$: if Tr executes $\mathsf{event}(\sigma M)$ then there are $\sigma'$ and $j$ with $\sigma' M =_E \sigma M$ such that Tr executes $\mathsf{event}(\sigma' M_{jk})$ for every $k \leq l_j$.

Fix $\mathcal{N}_{\mathrm{priv}}$ as in Definition 2.1, and for an $\mathcal{N}_{\mathrm{pub}}$-adversary $Q$ write $\mathcal{C}_Q$ for the configuration $(\mathcal{N}_{\mathrm{pub}}, \mathcal{N}_{\mathrm{priv}}), \{P_0, Q\}$.

**Definition 2.2** (Satisfaction)**.** A trace *satisfies* $\mathsf{attacker}(M)$ if it outputs $M$ publicly, and *satisfies* $\mathsf{event}(M)$ if it executes $\mathsf{event}(M)$.

**Definition 2.3** (Correspondences between facts)**.** Let $F$ and $H_{jk}$ ($j \leq m$, $k \leq l_j$) be query facts over $X$. $P_0$ *satisfies* $F \rightsquigarrow \bigvee_{j=1}^{m} \bigwedge_{k=1}^{l_j} H_{jk}$ *against* $\mathcal{N}_{\mathrm{pub}}$*-adversaries* if for every $\mathcal{N}_{\mathrm{pub}}$-adversary $Q$, every trace Tr from $\mathcal{C}_Q$ and every substitution $\sigma$: if Tr satisfies $\sigma F$ then there are $\sigma'$ and $j$ with $\sigma' F =_E \sigma F$ such that Tr satisfies $\sigma' H_{jk}$ for every $k \leq l_j$. For event facts this is Definition 2.1(4).

## 2.2 The correspondence

Throughout, fix a model (Definition 1.8) satisfying

(R1) Fr is the set of symbols at the root of no equation and $(E, \mathrm{Fr})$ satisfies (H);

(R2) every rule of every destructor has patterns over Fr, and the destructor is the one of Definition 1.2;

(R3) Pub consists of the public constructors and the names in $\mathcal{N}_{\mathrm{pub}}$, and Dt of the public destructors;

(R4) $P_0$ is a closed process of Definition 1.7.

These conditions exclude some ProVerif models: an equation with a variable as one side, such as the survey's (2.8), violates (H), and a destructor whose rules match on a symbol at the root of an equation has no counterpart under (R2).

Names of the reduction semantics are the free names together with the created names; terms are interpreted in $\mathcal{M}$ through a bijection $\iota$ between the names outside $\mathcal{N}_{\mathrm{pub}} \cup \mathcal{N}_{\mathrm{priv}}$, those that (Red Res) may create, and $\mathbb{N}$, extended by $\iota a = a$ for free names, $\iota a' = \nu_{\iota a'}$ for created names, $\iota f(\vec{M}) = f(\iota \vec{M})$ and $\iota\, \mathsf{fail} = \bot$; $\iota M = \iota M'$ exactly when $M =_E M'$. Let $\rho_\iota$ be the environment with $\rho_\iota(a) = \iota a$ for every free or created name $a$.

**Lemma 2.4** (Evaluation)**.** *For every closed expression $D$ over free and created names, some $U$ satisfies $D \Downarrow U$, and $D \Downarrow U$ implies $\iota U = [\![D]\!]\rho_\iota$.*

**Definition 2.5** (Labelled semantics)**.** A *labelled configuration* is $(U, \mathcal{P})$ with $U \subseteq \mathbb{N}$ finite and $\mathcal{P}$ a finite multiset of closed processes whose names are free names and symbols $\nu_k$. The $\tau$-steps $(U, \mathcal{P}) \xrightarrow{\tau} (U, \mathcal{P}')$ are the steps (Red Nil), (Red Par), (Red Repl), (Red Eval 1), (Red Eval 2),

(Red Cond 1′) and (Red Cond 2′), with $=_E$ read on $\mathcal{M}$. The visible steps are

$$(U, \mathcal{P} \cup \{\mathsf{out}(N, M); P\}) \xrightarrow{\mathsf{out}([N],[M])} (U, \mathcal{P} \cup \{P\}),$$

$$(U, \mathcal{P} \cup \{\mathsf{in}(N, x); P\}) \xrightarrow{\mathsf{in}([N],[M])} (U, \mathcal{P} \cup \{P\{M/x\}\}) \quad \text{for every closed term } M,$$

$$(U, \mathcal{P} \cup \{\mathsf{out}(N, M); Q,\ \mathsf{in}(N', x); P\}) \xrightarrow{\mathsf{comm}([N],[M])} (U, \mathcal{P} \cup \{Q,\ P\{M/x\}\}) \quad \text{if } N =_E N',$$

$$(U, \mathcal{P} \cup \{\mathsf{new}\ a; P\}) \xrightarrow{\mathsf{new}(k)} (U \cup \{k\}, \mathcal{P} \cup \{P\{\nu_k/a\}\}) \quad \text{for every } k \notin U,$$

$$(U, \mathcal{P} \cup \{\mathsf{event}(M); P\}) \xrightarrow{\mathsf{ev}([M])} (U, \mathcal{P} \cup \{P\}).$$

The third step is (Red I/O′) between two members of $\mathcal{P}$. It is visible rather than a $\tau$-step because a trace records a completed communication, which is what known reads the attacker's knowledge off. With $\stackrel{\alpha}{\Longrightarrow} = \stackrel{\tau}{\rightarrow}^* \stackrel{\alpha}{\rightarrow} \stackrel{\tau}{\rightarrow}^*$, the set of *weak traces* is

$$\mathrm{Tr}(U, \mathcal{P}) = \{\alpha_1 \cdots \alpha_n \in \mathrm{Tr} \mid (U, \mathcal{P}) \stackrel{\alpha_1}{\Longrightarrow} \cdots \stackrel{\alpha_n}{\Longrightarrow}\}.$$

**Theorem 2.6** (Adequacy)**.** *For every closed process $P$ whose names are free names and symbols $\nu_k$, $[\![P]\!]\rho = \mathrm{Tr}(\emptyset, \{P\})$, where $\rho$ extends $\rho_0$ by $\rho(\nu_k) = \nu_k$. For all finite multisets $\mathcal{P}, \mathcal{Q}$ of such processes, $\mathrm{Tr}(\emptyset, \mathcal{P} \uplus \mathcal{Q}) = \mathrm{Tr}(\emptyset, \mathcal{P}) \mid \mathrm{Tr}(\emptyset, \mathcal{Q})$.*

Let $\mathcal{A}_0 = \mathrm{Attacks}([\![P_0]\!]\rho_0)$. For a trace Tr from $\mathcal{C}_Q$ let $\mathrm{outputs}(\mathrm{Tr}) \subseteq \mathcal{M}$ be the set of $\iota M$ for the terms $M$ that Tr outputs publicly, and for $t \in \mathrm{Tr}$ say that Tr and $t$ *have the same events* if Tr executes $\mathsf{event}(M)$ exactly when $\mathsf{ev}(\iota M)$ occurs in $t$.

**Theorem 2.7** (Attacks)**.** *Fix an $\mathcal{N}_{\mathrm{pub}}$-adversary $Q$ and a trace* Tr *from $\mathcal{C}_Q$, as well as a trace $t \in \mathcal{A}_0$.*

*(E) (i) There is $t' \in \mathcal{A}_0$ with the same events as* Tr*.*

*(ii) There are an $\mathcal{N}_{\mathrm{pub}}$-adversary $Q'$ and a trace* Tr′ *from $\mathcal{C}_{Q'}$ with the same events as $t$.*

*(K) Suppose further $\mathcal{N}_{\mathrm{pub}} \neq \emptyset$.*

*(i) For every $m \in \mathrm{outputs}(\mathrm{Tr})$ there is $t'$ as in (E)(i) such that $m \in \mathrm{known}(\langle\emptyset\rangle, t')$.*

*(ii) For every $m \in \mathrm{known}(\langle\emptyset\rangle, t)$ there are $Q'$ and* Tr′ *as in (E)(ii) such that $m \in \mathrm{outputs}(\mathrm{Tr}')$.*

**Theorem 2.8** (Secrecy)**.** *Let $\mathcal{N}_{\mathrm{pub}} \neq \emptyset$, let $M$ be a closed term with $\mathrm{fn}(M) \subseteq \mathcal{N}_{\mathrm{pub}} \cup \mathcal{N}_{\mathrm{priv}}$ and let $s = \rho_0 M$. Then $\mathrm{Secret}([\![P_0]\!]\rho_0, s)$ holds if and only if $P_0$ preserves the secrecy of $M$ from $\mathcal{N}_{\mathrm{pub}}$.*

*Remark* 2.9. The hypothesis $\mathcal{N}_{\mathrm{pub}} \neq \emptyset$ cannot be dropped: when $\mathcal{N}_{\mathrm{pub}} = \emptyset$ no trace outputs anything publicly, so $P_0$ preserves the secrecy of every $M$, whereas $\mathrm{Secret}([\![P_0]\!]\rho_0, \rho_0 M)$ fails whenever $\emptyset \vdash \rho_0 M$, for instance for a public constant $M$.

**Theorem 2.10** (Correspondences)**.** *Let $F$ and $H$ be query facts over $X$ such that $H$ is an event fact, and let $\mathcal{N}_{\mathrm{pub}} \neq \emptyset$ if $F$ is an attacker fact. Then $P_0$ satisfies $F \rightsquigarrow H$ against $\mathcal{N}_{\mathrm{pub}}$-adversaries if and only if $\mathrm{Corr}([\![P_0]\!]\rho_0,\ \sigma \mapsto \sigma F,\ \sigma \mapsto \sigma H)$.*

*Remark* 2.11. Neither hypothesis can be dropped. When $\mathcal{N}_{\text{pub}} = \emptyset$ and $F = \mathsf{attacker}(M)$, $P_0$ satisfies the correspondence vacuously, whereas Corr fails as soon as $\emptyset \vdash \sigma M$ for some valuation $\sigma$. For $H = \mathsf{attacker}(M)$, Definition 2.2 requires $\sigma M$ itself to be output publicly, whereas $\mathsf{knows}(\sigma M)$ holds as soon as $\sigma M$ is derivable: with the symmetric encryption of the survey's §2.1, $\mathcal{N}_{\text{pub}} = \{c\}$ and $P_0 = \mathsf{out}(c, \mathsf{senc}(s, k)); \mathsf{out}(c, k); \mathsf{event}(e)$, Corr holds for $\mathsf{event}(e) \rightsquigarrow \mathsf{attacker}(s)$ while the correspondence of Definition 2.3 fails. The reading by derivability is ProVerif's.

## References


[1] Bruno Blanchet. Modeling and verifying security protocols with the applied pi calculus and ProVerif. *Foundations and Trends in Privacy and Security*, 1(1–2):1–135, October 2016. `../Reference/BlanchetFnTPS16.pdf`, doi:10.1561/3300000004.